

APUNTE DE ESTUDIO · BASES DE DATOS

Normalización de Bases de Datos

*Formas normales, dependencias funcionales
y método práctico de resolución*

Contenidos:

Concepto de normalización · Dependencias funcionales · Clave primaria (simple y compuesta)
Primera Forma Normal (1FN) · Segunda Forma Normal (2FN) · Tercera Forma Normal (3FN)
Diferencia entre 2FN y 3FN · Ejemplo completo resuelto · Método práctico paso a paso

1. ¿Qué es la normalización?

La **normalización** es un proceso que se utiliza para organizar los datos de una base de datos de manera que se reduzca la redundancia y se eviten problemas al insertar, modificar o eliminar información.

La idea general es partir de una estructura grande, donde hay datos mezclados y repetidos, e ir separándola progresivamente en tablas más pequeñas y relacionadas entre sí.

Objetivo real: no es simplemente "hacer muchas tablas", sino lograr que cada dato esté almacenado en el lugar que realmente le corresponde.

¿Qué problemas busca evitar?

Supongamos una tabla así, con datos de pedidos y artículos mezclados en una sola estructura:

Nro_Pedido	Cliente	Artículo	Descripción
100	Juan Pérez	10	Teclado
100	Juan Pérez	20	Mouse
101	María Gómez	10	Teclado

En rojo: datos repetidos (redundancia) que la normalización busca eliminar.

Figura 1. Tabla sin normalizar: el cliente y la descripción del artículo se repiten en varias filas.

Tenemos información repetida: "Juan Pérez" aparece varias veces si realizó un pedido con varios artículos, y "Teclado" podría repetirse cientos de veces cada vez que alguien compre el artículo 10.

Consecuencia: esto puede producir inconsistencias. Si cambia la descripción del artículo 10, habría que modificar muchas filas. Si alguna queda sin modificar, tendríamos información contradictoria. La normalización busca evitar justamente estos problemas.

2. Dependencias funcionales

Las **dependencias funcionales** son fundamentales para normalizar. Una expresión como:

Cod_articulo → Descripcion_articulo

significa: "conociendo el código del artículo, puedo determinar su descripción". Si el código identifica un único artículo, su descripción depende de él.

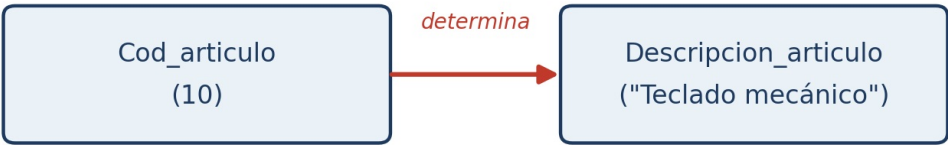


Figura 2. El código de artículo determina de forma única su descripción.

Otro ejemplo: Nro_cliente → Nombre_cliente significa que el número de cliente determina quién es ese cliente.

Las dependencias funcionales nos permiten detectar qué atributos pertenecen realmente juntos.

Clave primaria

La **clave primaria (PK)** es el atributo, o conjunto de atributos, que identifica de manera única cada registro.

CLIENTE
Nro_cliente PK
Nombre_cliente

PEDIDO ARTICULO
Nro_pedido PK
Nro_articulo PK
Cantidad

En el segundo caso la PK es compuesta: `(Nro_pedido, Nro_articulo)`, porque necesitamos ambos datos para identificar una línea del pedido. Por ejemplo, "Pedido 100 + Artículo 20" representa un artículo específico dentro de un pedido específico.

La existencia de una PK compuesta es especialmente importante cuando analizamos la **2FN**.

1FN Primera Forma Normal

El primer paso consiste principalmente en **detectar estructuras o grupos repetitivos**. Por ejemplo, imaginemos la siguiente estructura inicial:

Nro_pedido
Fecha_pedido
Nro_cliente
Nombre_cliente
Nro_articulo
Descripcion_articulo
Cantidad

Sabemos que un pedido puede tener varios artículos. Entonces `Nro_articulo`, `Descripcion_articulo` y `Cantidad` forman parte de una estructura que se puede repetir para un mismo pedido.

Por eso separamos en dos tablas:

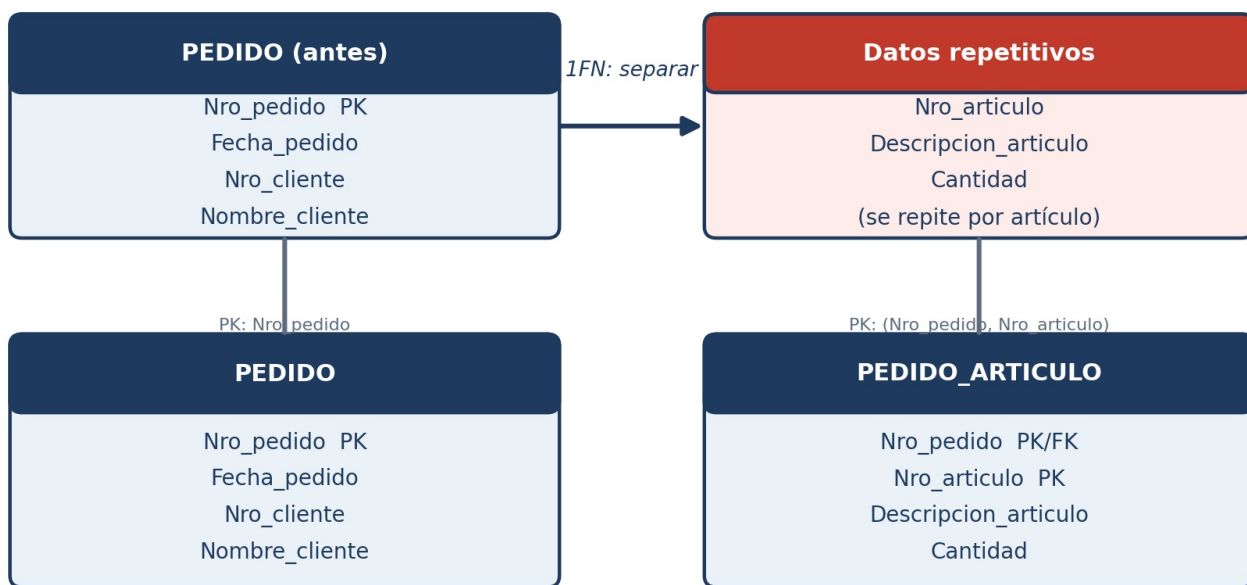


Figura 3. Aplicación de la 1FN: se separa el grupo repetitivo de artículos en una tabla propia.

La segunda tabla tiene como PK `(Nro_pedido, Nro_articulo)`. Cada registro representa: "este artículo pertenece a este pedido".

Idea práctica para 1FN: buscar datos que puedan repetirse varias veces dentro de un mismo registro principal.

2FN Segunda Forma Normal

Para analizar la 2FN hay que prestar especial atención a las **claves primarias compuestas**. La pregunta fundamental es:

¿Existe algún atributo que dependa solamente de **una parte** de la PK compuesta? Si ocurre, tenemos una **dependencia parcial**.

Volvamos al ejemplo de `PEDIDO ARTICULO`, cuya PK es `(Nro_pedido, Nro_articulo)`. Pero tenemos:

`Nro_articulo → Descripcion_articulo`

Para conocer la descripción del artículo no necesito conocer el número de pedido. Por ejemplo, si `Nro_articulo = 20`, la descripción es "Mouse inalámbrico" — da igual si aparece en el pedido 100, 200 o 500. Entonces `Descripcion_articulo` depende solamente de una parte de la PK

compuesta: es una **dependencia parcial**.

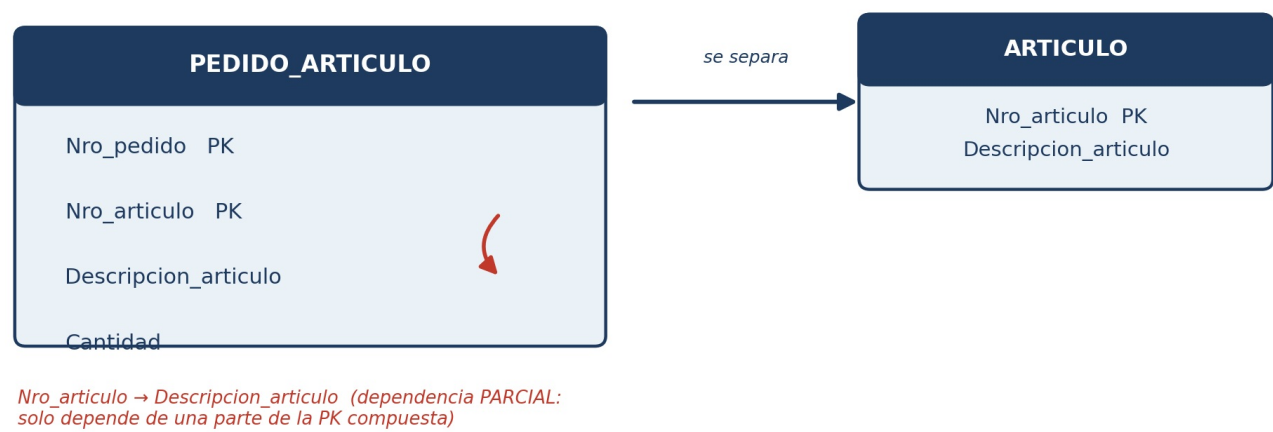


Figura 4. Dependencia parcial: la descripción depende solo de Nro_articulo, por lo que se separa en una tabla ARTICULO.

Ahora observemos **Cantidad**: tenemos $(\text{Nro_pedido}, \text{Nro_articulo}) \rightarrow \text{Cantidad}$. Esto está bien, porque para conocer la cantidad necesitamos saber qué cantidad de ese artículo fue solicitada en ese pedido puntual — no alcanza con conocer solamente el pedido ni solamente el artículo. Por eso **Cantidad** depende de toda la PK compuesta y **permanece** en **PEDIDO_ARTICULO**.

Regla práctica de 2FN: ante una PK compuesta (A, B), preguntate para cada atributo si depende de A y B juntos. Si depende solamente de A o solamente de B, probablemente tengas que separarlo.

En la 3FN buscamos las llamadas **dependencias transitivas**. La idea es detectar situaciones como:

```
A → B
B → C
Por lo tanto: A → B → C
```

El atributo C no depende realmente de A: depende de B. Veamos el ejemplo de `PEDIDO`, donde tenemos `Nro_pedido → Nro_cliente` y `Nro_cliente → Nombre_cliente`.

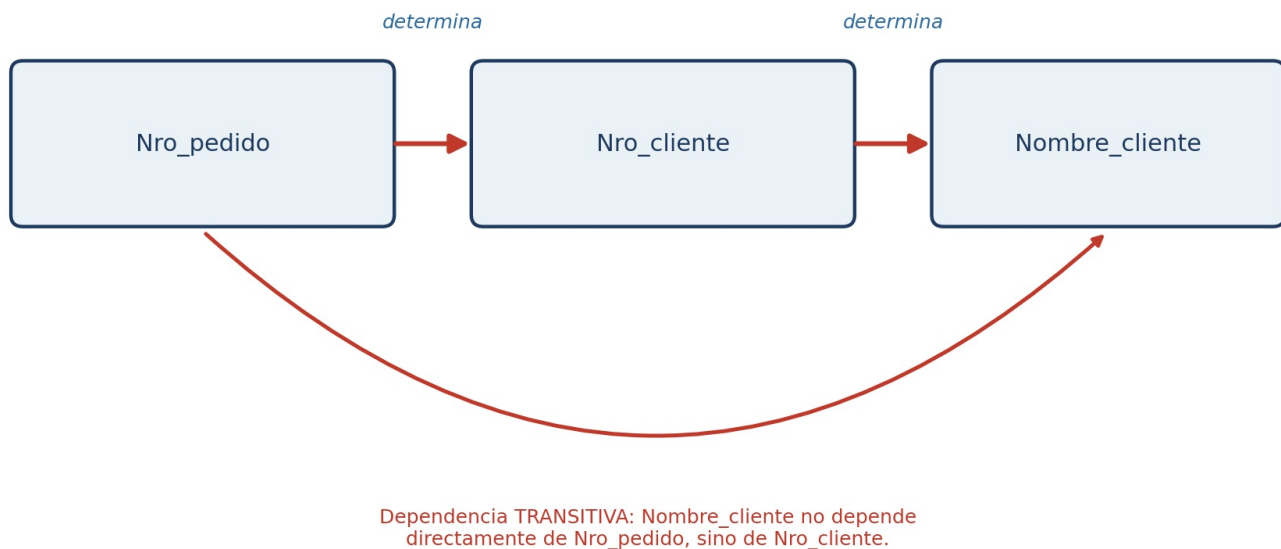


Figura 5. Cadena de dependencia transitiva: Nombre_cliente depende de Nro_cliente, no del pedido.

`Nombre_cliente` no es realmente un dato del pedido: es un dato del cliente. Por eso se crea una tabla `CLIENTE` y en `PEDIDO` queda solamente `Nro_cliente` como clave foránea (FK).

Otro ejemplo de dependencia transitiva

En el ejercicio de artículos con marca, tenemos: `Cod_articulo → Cod_marca_articulo` y `Cod_marca_articulo → Nombre_marca`. Como `Nombre_marca` depende realmente del código de marca, se normaliza creando una tabla `MARCA` separada. Lo mismo ocurre con `Cod_pieza → Cod_pais_origen → Nombre_pais`, que da lugar a una tabla `PAIS_ORIGEN`.

Diferencia clave entre 2FN y 3FN

Esta es probablemente la parte más importante para el examen:

2FN

El problema aparece porque un atributo depende de **una parte** de una PK **compuesta**.

```
PK = (Cod_articulo, Cod_pieza)
Cod_pieza → Nombre_pieza
→ dependencia parcial.
```

3FN

El problema ocurre entre atributos que **no son parte de la PK** compuesta.

```
Cod_pieza → Cod_pais
Cod_pais → Nombre_pais
→ dependencia transitiva.
```

3. Ejemplo completo resumido

Partimos de la siguiente estructura, sabiendo que un pedido puede contener muchos artículos:

Nro_Pedido, Fecha_Pedido, Nro_Cliente, Nombre_Cliente, Nro_Articulo, Descripcion_Articulo, Cantidad

Paso 1FN — separar el grupo repetitivo

<u>PEDIDO</u> Nro_Pedido PK Fecha_Pedido Nro_Cliente Nombre_Cliente	<u>PEDIDO ARTICULO</u> Nro_Pedido PK/FK Nro_Articulo PK Descripcion_Articulo Cantidad
---	---

Paso 2FN — sacar la dependencia parcial

Nro_Articulo → Descripcion_Articulo es una dependencia parcial. Se crea ARTICULO :

<u>ARTICULO</u> Nro_Articulo PK Descripcion_Articulo	<u>PEDIDO ARTICULO</u> Nro_Pedido PK/FK Nro_Articulo PK/FK Cantidad
--	--

Paso 3FN — sacar la dependencia transitiva

En PEDIDO : Nro_Pedido → Nro_Cliente → Nombre_Cliente . Se crea CLIENTE :

<u>CLIENTE</u> Nro_Cliente PK Nombre_Cliente
--

Resultado final — modelo normalizado en 3FN

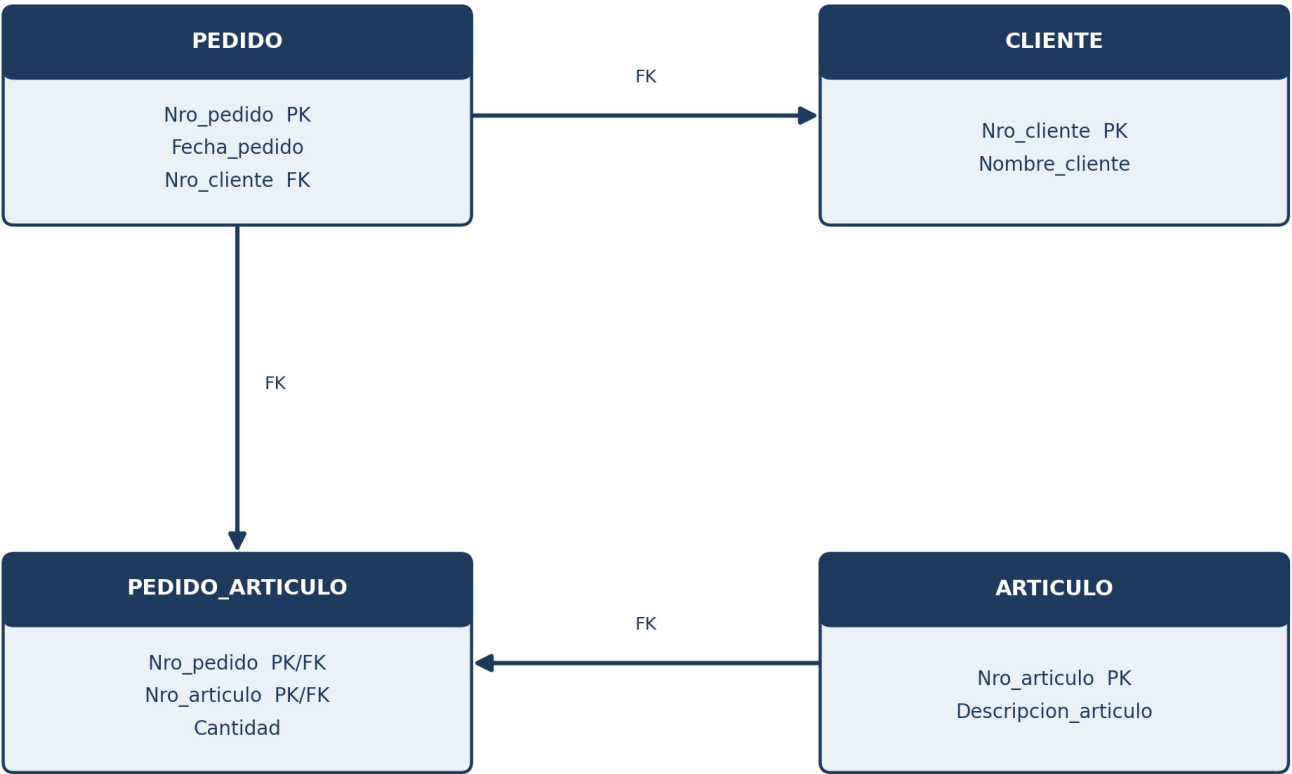


Figura 6. Las cuatro tablas resultantes, relacionadas mediante claves foráneas (FK).

4. Método práctico para resolver ejercicios

Cuando te den una estructura para llevar a 3FN, podés seguir siempre esta secuencia:

1. **Identificar la PK inicial** y leer cuidadosamente el enunciado. Determinar qué representa cada registro y qué datos pueden repetirse.
2. **1FN**: buscar estructuras repetitivas. Si algo puede aparecer varias veces para una misma entidad principal, probablemente necesites una estructura separada.
3. **Identificar las PK compuestas** que aparezcan.
4. **2FN**: buscar dependencias parciales. Si la PK es (A, B), revisar si algún atributo depende solamente de A o solamente de B.
5. **3FN**: buscar dependencias transitivas. Buscar cadenas como $A \rightarrow B \rightarrow C$. Si C depende realmente de B, crear una nueva tabla.
6. **Marcar correctamente PK y FK**. Las PK identifican registros y las FK mantienen las relaciones entre las tablas.

La idea que conviene memorizar

- **1FN**: saco grupos o estructuras repetitivas.
- **2FN**: saco atributos que dependen solamente de una parte de una PK compuesta.
- **3FN**: saco atributos que dependen de otros atributos que no son la PK.

Forma conceptual de pensarlo: **cada atributo debe quedar almacenado junto a aquello que realmente lo determina.**

Nombre_marca pertenece a MARCA · Nombre_cliente pertenece a CLIENTE · Descripción_articulo pertenece a ARTICULO · Cantidad pertenece a la relación entre PEDIDO y ARTICULO.

Cuando empezás a pensar los ejercicios de esta manera, la normalización deja de ser tan mecánica y se vuelve bastante más intuitiva.